

Hexa Team present...

Data Science Final Project

Members and Roles :

Ali Mohamed Fathy “ The Leader “

- Data Cleaning*
- Core Algorithms*
- Data Visualization*
- GUI*

Omar Gaber

- Data Cleaning*
- K-means Algorithm*

Omar Hossam

- Data Visualization*

Ammar Mahmoud

- GUI*

Data Cleaning

1. Introduction:

This report explains the process that happened to make the Row Data usable in analysis and inference so we went through several stages to do this .

2. Inspect the dataset structure and data types:

In this stage we use some of functions like “str()” and “summary()” to know what is the data that we work with .

2.1. Changing Data Types

we start to change the Data types which requires this , like from numeric to integer , from character to factor and in this case we used function “ unique()” to know all value in the columns

2.2. Remove the Rows that have spaces in the cells

when we used “unique()” function we conclude that we have Rows have just spaces in cells so we deleted it by “Filter()” Function

the code :

```
Data_Cleaned <- Data_Cleaned %>%  
  filter( Weather != "" &  
          Traffic_Level != "" &  
          Time_of_Day != "" )
```

2.3. Give levels to the character

we gived levels to the Values in the columns that Which we converted to factor

Example Code :

```
Data_Cleaned$Traffic_Level <- factor(  
  c("Low", "Medium", "High"),  
  levels = c("Low", "Medium", "High"),  
  ordered = TRUE  
)
```

3. Find Outliers and Delete it :

In this stage we found the Outliers Which mean the Value that is Unexpected and significantly different from the other existing values and Delete it by using the Boxblot

The code :

```
boxplot(Data_Cleaned[,2:9])$out  
outlier <- boxplot(Data_Cleaned$Delivery_Time_min)$out  
Data_Cleaned <- Data_Cleaned[- which(Data_Cleaned$Delivery_Time_min %in% outlier),]
```

4. Arrange the Data and Structure it.

arrange the DataFrame ID's by using "arrange()" Function and Shifting the ID's by using loop.

The code for arrange :

```
Data_Cleaned <- Data_Cleaned %>%  
  arrange(Order_ID)
```

The code for Shifting :

```
for( i in 1 : length(Data_Cleaned$Order_ID)){  
  Data_Cleaned[i,1] <- i  
}
```

5. Adding New Columns “Future Engineering”:

In this case we wanted to make new columns in our Data Frame to help us then in analysis and understanding new relationships, clearly identifying problems, and also in using columns that might not be useful in the analysis if we didn't create those new columns.

5.1. Speed Column

Therefore, the speed column was created, which links distance and time together and makes us able to compare orders and thus identify which orders were delayed.

We make columns by using “mutate()” function

The code :

```
Data_Cleaned <- Data_Cleaned %>%  
  mutate(Speed_KPH = ( Data_Cleaned$Distance_km / Data_Cleaned$Delivery_Time_min ) * 60)
```

5.2.customer rating Column

we calculate it to know is the service good to the users

the code :

```
max_speed <- max(Finished_Data_Cleaned$Speed_KPH)
min_speed <- min(Finished_Data_Cleaned$Speed_KPH)

Level_of_Speed <- (max_speed-min_speed)/5

Finished_Data_Cleaned$Customer_Rating <- NA

for ( x in 1:5 ){

  lower_bound <- min_speed + (x - 1) * Level_of_Speed
  upper_bound <- min_speed + x * Level_of_Speed

  if (x == 5) {
    Rate_col_fit <- Finished_Data_Cleaned$Speed_KPH >= lower_bound &
    Finished_Data_Cleaned$Speed_KPH <= upper_bound
  } else {
    Rate_col_fit <- Finished_Data_Cleaned$Speed_KPH >= lower_bound &
    Finished_Data_Cleaned$Speed_KPH < upper_bound
  }
  Finished_Data_Cleaned$Customer_Rating[Rate_col_fit] <- x
}
```

6. NA Values

Instead of Deleting these Rows , we choose to process them using the KNN Algorithm, which works as follows:

1. This Algorithm works with numbers, so all factors must be converted to numerical Data Type .
2. This Algorithm searches for the closest rows that correspond to the row that we should fill in the NA with.

The code :

```
#Handle the NA

my_features <- c("Weather", "Traffic_Level", "Time_of_Day", "Vehicle_Type","Speed_KPH")
New_Data_Cleaned <- Data_Cleaned

for ( y in 3:6){
  New_Data_Cleaned[,y] <- as.numeric(New_Data_Cleaned[,y])
}

Finished_Data_Cleaned <- kNN(

  data = New_Data_Cleaned,
  variable = "Courier_Experience_yrs",
  dist_var = my_features,
  k = 5,
  numFun = median,
  imp_var = FALSE,
)
```

K-means Clustering Analysis :

1. Introduction:

This report explains the process of using K-Means Clustering to divide our data into small groups .The main goal is to find the best group and the weakest group by looking at their average scores. Once we find the weakest group, we can understand its problems and plan ways to fix them to improve performance.

2. Preparing the Data (Data Pre-processing)

2.1. Dealing with Different Data Types

The Problem: The K-Means method works best only with numbers. Our data included non-useful information like ID numbers and YES/NO answers (Boolean data), as well as text categories (Factors).

The Solution: We removed the ID and Boolean column. We also changed the text categories into numbers by giving them an order (like 1, 2, 3) so they can be used in the math part of the analysis.

The Function: Basic data cleaning steps were performed to prepare the data.

2.2. Making Numbers at The Same Range

The Code:

```
Clusters_Data = Finished_Data_Cleaned[,-c(1,12)]
```


The Problem: Some numbers in our columns were very large ,and others were very small. This difference can make the big numbers unfairly control the clustering result.

The Solution: We scaled the data. Scaling means making all the numbers have a similar range. This helps the algorithm treat every column equally and gives us a more accurate result.

The Function: We used the scale() function.

The Code:

```
Scaled_Data = scale(Clusters_Data)
```

3. Finding the Best Number of Groups (K).

3.1. Choosing the Right K

The Problem: We need to find the best number of groups (K) to use. We want a number that gives us the smallest error inside each group without making the model too complicated.

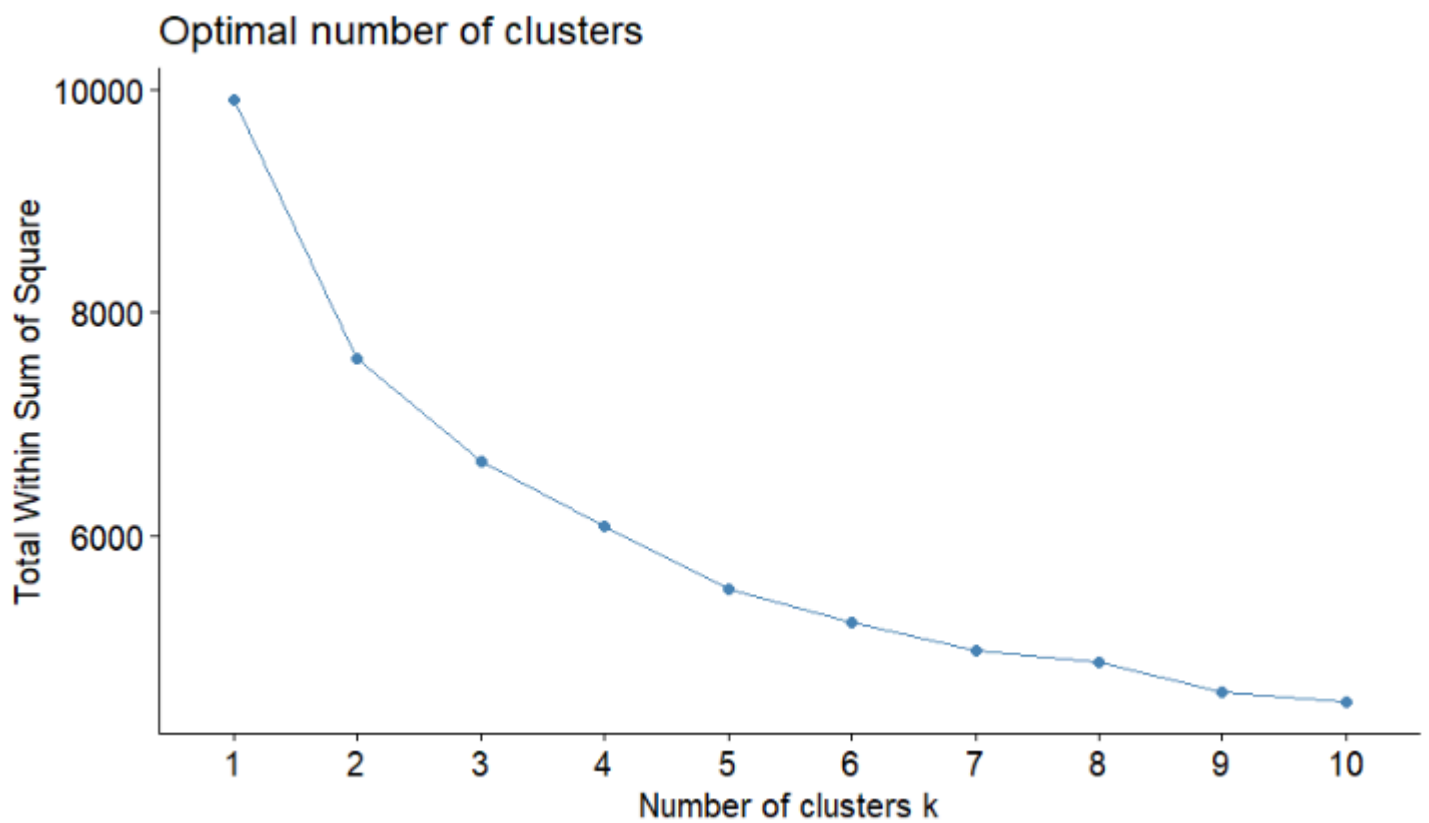
The Solution: We used the Elbow Method. This method shows us a chart where we look for the "elbow point"—the place where the error stops dropping quickly. This point is often the best K, which was 3 in our data.

The Function: We used the fviz_nbclust function.

The Code:

```
fviz_nbclust(Scaled_Data , kmeans,method ="wss")
```

The Chart:



4. Running the K-Means Algorithm

4.1. Applying the Grouping Method

The Problem: We need to run the K-Means algorithm to actually divide the data into the 3 groups we chose.

The Solution: We ran the K-Means function and used a setting called nstart. This setting is very important because K-Means starts randomly.

by using `nstart`, the computer runs the grouping process many times and then chooses the single best result (the one with the lowest error).

The Function: We used the `kmeans()` function.

The Code:

```
kdata3 = kmeans(Scaled_Data , centers = 3 , nstart =50)
```

4.2. Adding the Group Number

The Problem: We need to know which group (cluster) each data point belongs to.

The Solution: We added a new column to our numeric data table. This column adds the number of the cluster (1, 2, or 3) for every line of data.

The Code:

```
Clusters_Data$Clusters = kdata3$cluster
```

5. Analyzing and Comparing the Groups

5.1. Finding the Average Scores

The Problem: To compare the groups, we need to find the mean score for every feature in each cluster.

The Solution: We first grouped the data by the new cluster column. Then, we asked the system to calculate the average for all the features within those groups.

The Function: We used the `group_by()` function to put data into groups, and then the `summarise()` function, which uses the `mean()` function, to create a summary table of the averages.

The Code:

```
Cluster_Sammary = Clusters_Data %>%  
  group_by(Clusters) %>%  
  summarise(  
    Avg_DTime = mean( Delivery_Time_min),  
    Avg_Distance = mean(Distance_km),  
    Avg_speed = mean(Speed_KPH),  
    Avg_weather = mean(Weather),  
    Avg_Traffic = mean(Traffic_Level),  
    Avg_Time = mean(Time_of_Day),  
    Avg_Vehicle = mean(Vehicle_Type),  
    Avg_Rating = mean(Customer_Rating),  
    Avg_prep = mean (Preparation_Time_min),  
    Avg_Experience = mean (Courier_Experience_yrs),  
  )
```

5.2. Visualizing the Groups (Reducing Dimensions)

The Problem: Our data has 11 columns (11 dimensions), which is impossible to see in one chart. Computers can only show 2 dimensions well.

The Solution: We used Principal Component Analysis (PCA) to change our 11 columns of data into just 2 main columns (2D). This keeps most of the original data's information but makes it easy to see.

The Function: We used the `prcomp()` function.

The Code:

```
PCA_Data <- prcomp(Scaled_Data ,scale= FALSE , center = FALSE)
```

5.3. Drawing the Final Groups

The Problem: We need a picture to show how the 3 groups are separated on the 2D chart.

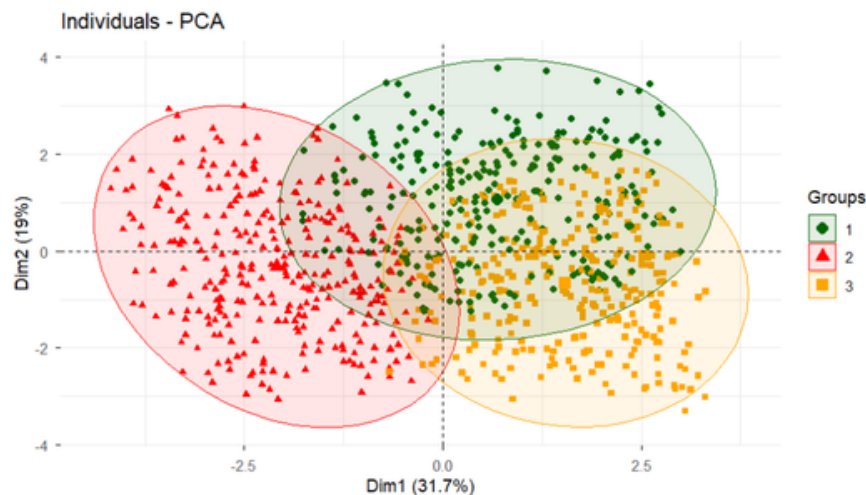
The Solution: We made a chart showing all the data points, and we colored each point based on its cluster number. We also drew an oval border around each group.

The Function: We used the `fviz_pca_ind()` function.

The Code:

```
fviz_pca_ind(PCA_Data,  
             col.ind = Clusters_Data$Clusters,  
             palette = c("darkgreen", "red", "orange"),  
             addEllipses = TRUE,  
             legend.title = "Groups",  
             geom = "point"  
)
```

The Chart :



5.4. Understanding the Chart Directions

The Problem: The picture shows us the groups, but we don't know what makes them different (we don't know what the directions on the chart mean).

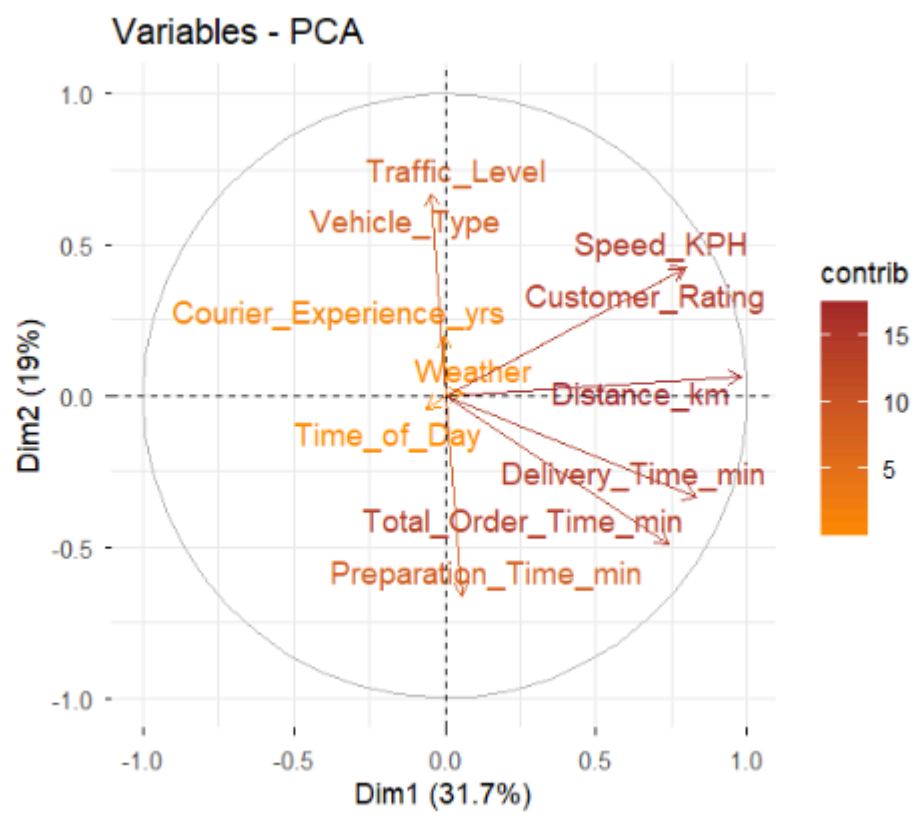
The Solution: We added arrows to the chart. These arrows represent the original features (columns). If a group is near a certain arrow, it means that group scores high on that feature. This helps us explain the meaning of each cluster.

The Function: We used the `fviz_pca_var()` function.

The Code:

```
fviz_pca_var(  
  PCA_Data,  
  col.var = "contrib",  
  gradient.col= c("darkorange", "brown"),  
  repel=TRUE  
)
```

The Chart:



Association Rules Analysis:

0-Introduction:

The objective of this section is to identify the main factors that contribute to delayed deliveries. To uncover these hidden patterns, association rules must be generated between the dataset variables. However, several challenges must be addressed before meaningful rules can be extracted.

1-Restoring original variable types:

~~The Problem:~~ Many variables were originally converted or encoded and ~~therefore needed~~ to be restored to their proper categorical values.

The Solution: Categorical variables were converted back to their original factor levels using the factor function.

The Functions: factor() function.

Code Sample:

```
# Traffic_Level (Col 4) - Original levels: "Low", "Medium", "High" (3 levels)
Finished_Data_Cleaned$Traffic_Level <- factor(
  Finished_Data_Cleaned$Traffic_Level,
  levels = 1:3,
  labels = c("Low", "Medium", "High"),
  ordered = TRUE
)
```


2-Discretizing numerical variables:

The Problem: Numerical variables cannot be used directly in association rule mining and must be transformed into categorical range.

The Solution: Numeric fields were transformed into meaningful categories (e.g., Low, Medium, High).

The Functions: cut() function.

Code Sample:

```
Finished_Data_Cleaned$Distance_Category <- cut(  
  Finished_Data_Cleaned$Distance_km,  
  breaks = c(0 , 5, 15, 20),  
  labels = c("Short", "Medium", "Long")  
)
```

3-Selecting relevant variables:

The Problem: Only a subset of variables is relevant for rule generation, and these must be selected carefully.

The Solution: A new dataset was created containing only the variables required for building association rule.

The Functions: Basic methods.

Code Sample:

```
data_cat <- Finished_Data_Cleaned[, c(  
  "Weather", "Traffic_Level", "Time_of_Day",  
  "Vehicle_Type", "Delivery_Category",  
  "Distance_Category", "Prep_Category",  
  "Exp_Category", "Late_Delivery", "Speed_Category",  
  "Order_Time_Category"  
)]
```

:Converting variables to factors .4

.The Problem: All variables used in association rules must be of type factor

.The Solution: All selected features were converted into factor type

.The Functions: as.factor() function

:The Code

```
data_cat <- as.data.frame(lapply(data_cat, as.factor))
```

:Creating transactions .5

The Problem: the dataset must be converted into transaction format before rule mining can be applied

The Solution: Transactions were generated to prepare the data for the apriori algorithm

.The Functions: as.transaction() function

:The Code

```
trans <- as(data_cat, "transactions")
```

6-Generating association rules:

The Problem: now we need to make rules to determine the relations between them

The Solution: Rules were produced specified support and confidence thresholds, then sorted by strength

The Functions: apriori () function.

The Code:

```
rules <- apriori(data = trans,
                 parameter = list(
                   supp = 0.01,
                   conf = 0.4,
                   minlen = 2,
                   maxlen = 3
                 ))
```

7-Extracting rules leading to delays:

The Problem: now we need to make rules to determine the relations between them

The Solution: Rules with Late_Delivery = TRUE on the RHS were isolated to focus on the patterns contributing to late deliveries.

The Functions: subset() function.

The Code:

```
late_rules <- subset(rules_sorted, rhs %in% "Late_Delivery=TRUE")
```

Data Visualization:

Introduction:

In this section we will discuss every plot :

- Why we made it
- What we concluded from it
- How to benefit from it by presenting some suggestions
- Plot image

Plot1(Best1):

~~-Reason:~~ We made best 1 a histogram to present the distribution of order time and geom density to present the density of total delivery time geom vlins to present specific values like mean and median

-Conclusions:

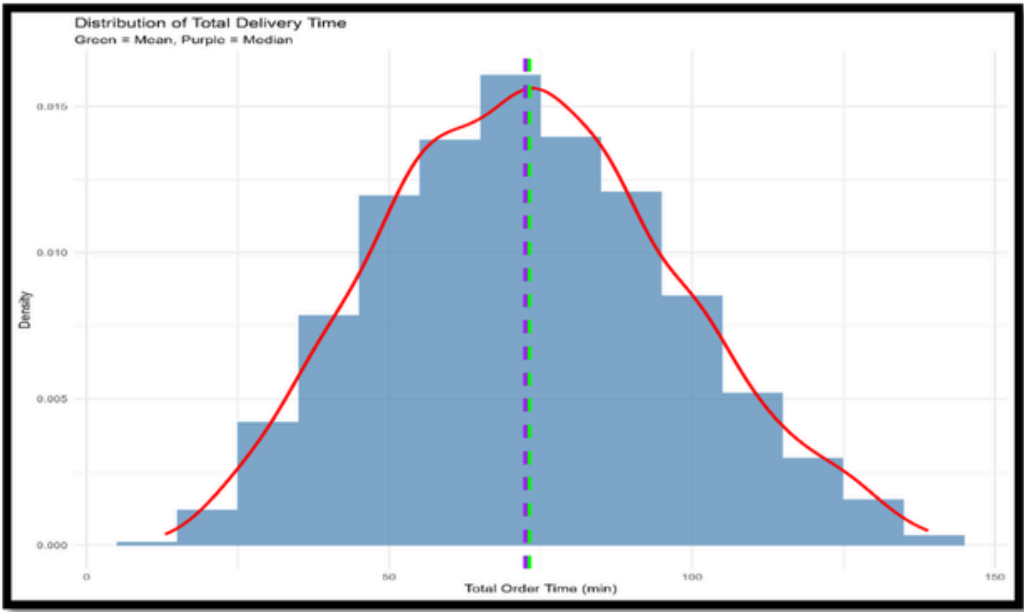
- The total time of the orders takes time from 50 to 100 (min).
- The mean is green line, and the median is purple line are almost the same.

So, the orders are late because the total time takes 75 (min)

-Suggestion:

- To decrease it we must decrease the preparation time of order or the time taken by the vehicle.

-Plot:



Plot2(Best2):

-Reason: We made Best2 violin plots to visualize the density and concentration of orders over time, and box plots to identify and remove outliers.

-Conclusions:

- That the bike took more time than the car and the scooter.
- That the scooter took the least time to deliver the order.
- That car is not as bad as bike, but it still takes more time than scooter

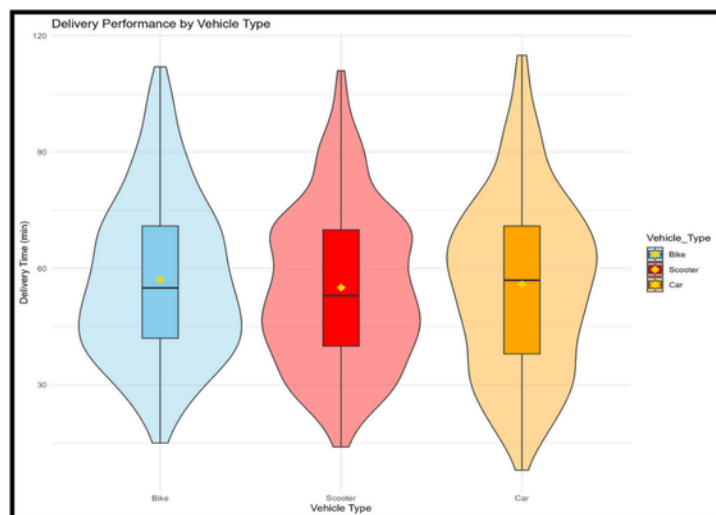
So, the vehicle that may cause the late of the orders is bike.

-Suggestion:

- To decrease late orders, we can sell some bikes and buy more scooters.

Note: “Maybe selling some cars will decrease it too but cars are useful for people who order many orders and good for far orders that may take a long time by scooter.”

-Plot:



Plot3(Best3):

-Reason: We made best3 geom points to see each rate alone in the plot and smooth to see the change in the relation between ratings and delivery time.

-Conclusions:

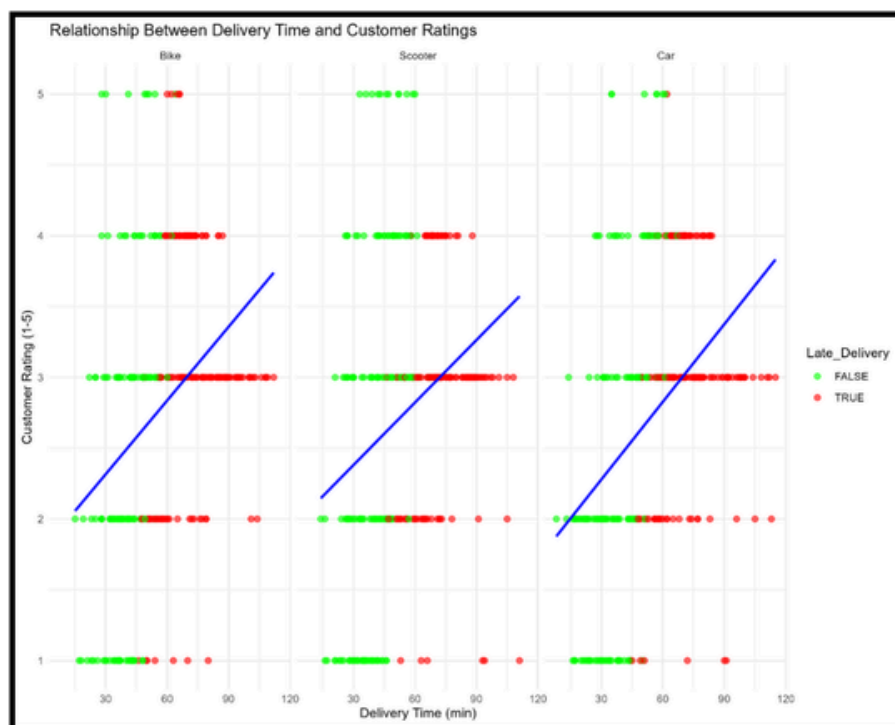
- The orders will be late when the delivery time take more than 60.

So, the vehicles are more related to the order than anything else.

-Suggestion:

- To increase customer ratings we can use appropriate vehicle according to distance

-Plot:



Plot4(Best4):

-Reason: We made best4 histogram plots to present the distribution of the numerical variables.

-Conclusions:

- The blue graph shows that some of orders are far and away from the store so we need a vehicle that can go too far easily like cars.

Note: “that is another reason for not selling cars!”

- The green graph shows that orders are almost all done from 0 to 10 (min) but few take time from 20 to 30 (min).

- The purple graph shows that there is no big difference in the experience years but still the experienced workers are fewer.

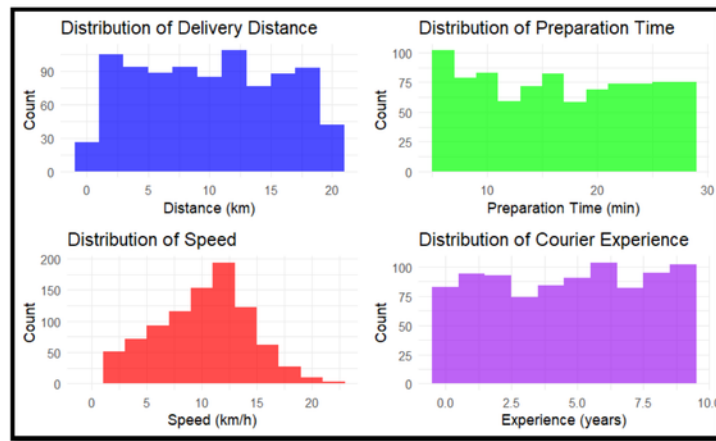
- The red graph shows that most drivers drive at a speed between 10 to 15 (km/h). So, the vehicles are more related to the order than anything else.

So, the graph may not show big differences by it appears in the graph of the experience of workers.

-Suggestion:

- Decrease the number of late orders by increasing the number of experienced workers.

-Plot:



Plot5(Best5):

-Reason: We made best5 a box plots geoms to present the outliers of the numerical variables across different categories.

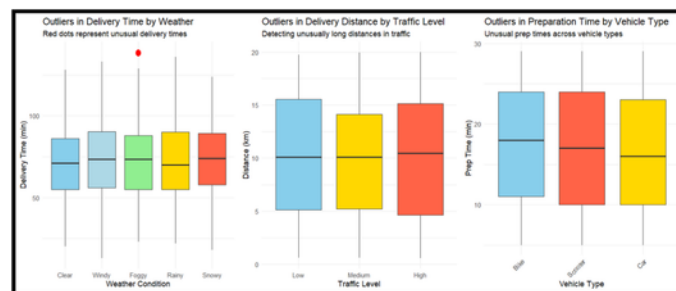
-Conclusions:

- There are a few outliers in the data after data cleaning

-Suggestion:

- Using cars in foggy weather and train our drivers to deal with dangerous weather

-Plot:



Plot6(Best6):

-Reason: We made best5 a box plots geoms to present the outliers of the numerical variables across different categories.

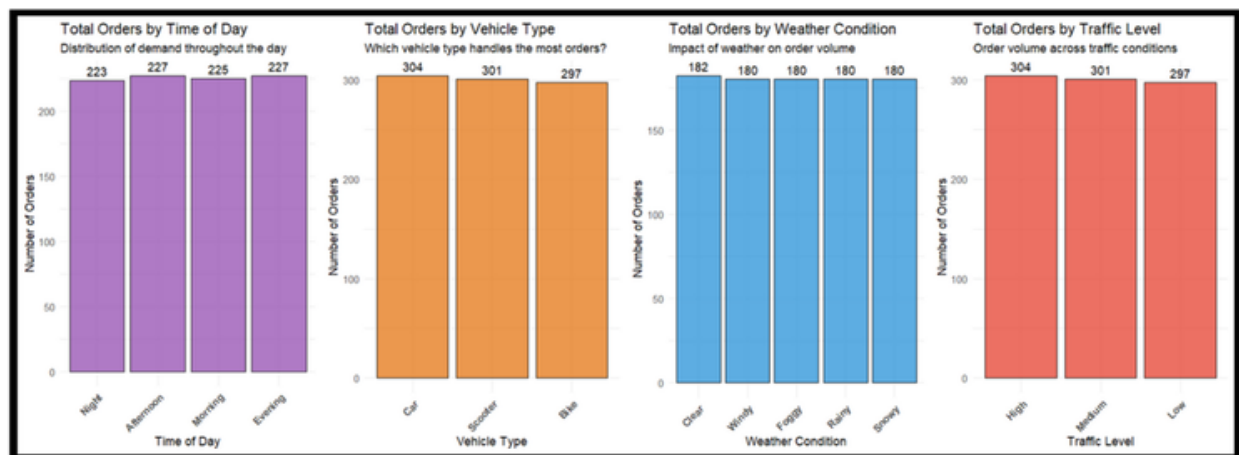
-Conclusions:

- The number of orders in the evening is highest .
- The number of orders delivered by car is highest.
- The weather conditions is almost the same.
- most of the orders in high traffic level

-Suggestion:

we should concentrate on making delivery by other vehicles better and concentrating on the orders in the rest on the day

-Plot:



Plot7(Best7):

-Reason: We made best7 a correlation matrix to present the relation between pairs of numerical variables.

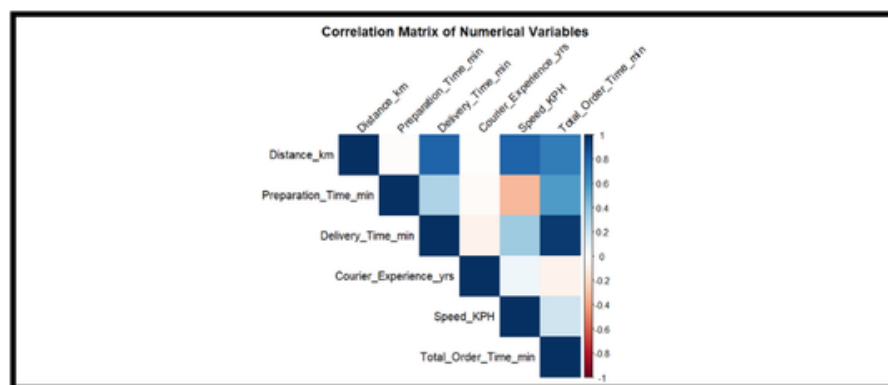
-Conclusions:

- There is a strong relationship between total time and distance.
- There is a bad relationship between speed and courier experience years.

-Suggestion:

- To decrease the late orders, we can tell the customer the right time of orders to arrive and so the order won't be late.

-Plot:



Plot8(Best8):

-Reason: We made best8 a geom density ridges to present the distribution of the delivery time across the of day.

-Conclusions:

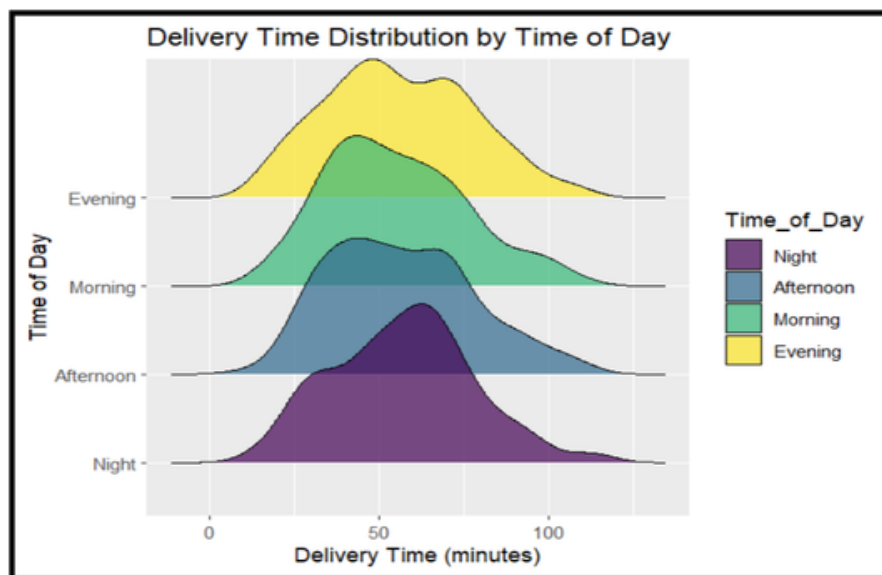
- The purple graph (night) shows that most orders take time from 50 to 75 (min)
- The green graph (morning) shows that most orders take from 25 to 50 (min) and decrease..

So, the delivery time in the morning is better than at night.

-Suggestion:

- To decrease the late orders, we can open the store in the morning to decrease the lateness of the orders.

-Plot:



Key Analysis Questions:

1- What are the primary factors that cause late deliveries?

After looking closely at all the factors that cause delays, our team found that the main reasons why orders are late are:

- Taking a long time to complete the order, including both kitchen preparation and delivery time.
- Long distance + slow driver speed makes the delivery time much longer.
- Even if kitchen preparation is not too long, heavy traffic also causes delays.

2- Are there any hidden patterns in delayed deliveries? (Using Association Rules)

To find hidden problems in late deliveries, we used association rules.

This analysis showed that delays are not caused by just one single thing. Instead, they happen because of strong and repeated connections between several problems that always come together. The most significant patterns we found are:

- If an order takes a long time overall, it will be late.
- Orders that take a long time (kitchen prep + travel) are often late.
- If the trip is long and the driver is slow, the order is likely to be late. This combination happens often in our data.
- Medium preparation time + high traffic leads to delays. Even a moderate kitchen time becomes a problem when traffic is bad.

3- Can delivery orders be grouped into meaningful clusters? What do these clusters represent?

We divided the data into 3 meaningful clusters. Each cluster represents a specific group with unique details:

Group 1 (The Best):

This group has the highest speed orders and the highest ratings.

It has only one problem: long delivery time. However, this is normal because of the long distance. **Solution:** We can solve this by batching orders (grouping them together) to save costs and focusing on efficiency for these long trips.

Group 2 (The Nightmare):

This group has lowest speed and the lowest ratings, even though it has the shortest distance. This group causes a big loss for the system, so we must solve its problems.

- **Solution:** It is better to use bikes or motorcycles in this group to avoid traffic and parking issues. Also, we should decrease the preparation time by cooperating better with restaurants.

Group 3 (The Challenging):

This group contains orders during dangerous weather. Despite that, the speed and ratings are good.

- **Solution:** It is better to use cars for this group. This will ensure employee safety and protect the food quality from the bad weather.

4- What specific actions can the company take to reduce delays?

Based on these problems, we can take several direct actions to reduce delays and make our work better:

1. Help Drivers Go Faster and Plan Better Routes

The long distance/slow driver problem is a main reason for delays. To fix this, we should:

- Use GPS tools to find the fastest roads for drivers.
- Watch driver speeds and give feedback if someone is always slow.
- Send experienced drivers on the longest trips.
- Move faster drivers to busy traffic areas when needed.

2. Change Delivery Zones and Distance Limits

Since long-distance orders cause a high risk of delays:

- We should check our delivery areas, especially during busy hours.
- Promise longer delivery times for customers who live far away.

3. Cut Down the Total Order Time

Because long prep and travel times strongly drive delays:

- We can make the kitchen work flow better so food is ready faster.
- Start rules to stop drivers from holding too many orders at once for too long.

4. Handle Heavy Traffic Better

Heavy traffic makes delays worse. To manage this, we should:

- Use live traffic maps to change delivery times.
- Increase the number of drivers available during high-traffic times.

5. Make Changes Based on Time of Day

Delays are often tied to certain busy hours:

- We need to hire more drivers during busy hours.

- Change staff work schedules to match when we get the most orders.

6. Give Customers Better Time Estimates

To make customers happier and reduce how bad delays feel:

- We can use past data and live traffic info to guess arrival times more accurately.
- If traffic gets worse or the kitchen is slow, we must update the arrival time right away.

7. Watch the Main Problems

Checking the main reasons for delays helps stop them before they happen:

- We should always check driver speed, kitchen time, distance, and traffic levels on a dashboard.
- Set up alerts when risky patterns appear (like a long trip combined with high traffic).

8. Choosing the Right Vehicle

We should not choose vehicles randomly. Instead, we must assign:

- bikes for short and crowded trips.
- cars for long trips or bad weather.